P1814R0
Mike Spertus, Symantec
Audience: Core Working Group

Wording for Class Template Argument Deduction for Alias Templates

This paper provides wording for the CTAD for alias templates section of P1021R4.

Document revision history

R0, 2019-07-19: Split from P1021R4

Wording

Modify (dcl.type.simple/2) as follows

> A *placeholder-type-specifier* is a placeholder for a type to be deduced (9.1.7.5). A *type-specifier* of the form `typename`$_{opt}$ *nested-name-specifier*$_{opt}$ *template-name* is a placeholder for a deduced class type (9.1.7.6). ~~The template-name shall name a class template.~~ The *nested-name-specifier*, if any, shall be non-dependent and the *template-name* shall name a deducible template. A deducible template is either a class template or an alias template whose *defining-type-id* is of the form `typename`$_{opt}$*nested-name-specifier*$_{opt}$`template`$_{opt}$ *simple-template-id* where the *nested-name-specifier* (if any) is non-dependent and the *template-name* of the *simple-template-id* names a deducible template. [Note: An *injected-class-name* is never interpreted as a *template-name* in contexts where class template argument deduction would be performed (13.7.1). — end note]

In over.match.class.deduct, modify the beginning of paragraph 1 as follows:

> When resolving a placeholder for a deduced class type (9.1.7.6) where the *template-name* names a primary class template of C a set of functions and function templates, called the guides of *C*, is formed comprising:

In over.match.class.deduct, append after paragraph 1 as follows:

> - The return type is the *simple-template-id* of the *deduction-guide*.
>
> When resolving a placeholder for a deduced class type (dcl.type.simple 9.1.7.2) where the *template-name* names an alias template `A` whose *defining-type-id* is a *simple-template-id* `B<L>`, the guides of `A` are the set of function or function templates formed as follows. For each function or function template `f` in the guides of `B`, form a function or function template `f'` according to the following procedure and add it to the set:
>
> - Deduce the template arguments of the return type of `f` from `B<L>` according to the process in (temp.deduct.type) with the exception that deduction does not fail if not all template arguments are deduced. Let `g` denote the result of substituting these deductions into `f`. If the substitution fails, no `f'` is produced.
> - Form the function or function template `f'` as follows:
>   - The function type of `f'` is the function type of `g`.
>   - If `f` is a function template, the template parameter list of `f'` consists of all the template parameters of `A` (including their default template arguments) that appear in the above deductions or (recursively) in their default template arguments, followed by the template parameters of `f` that were not deduced (including their default template arguments).
>   - The associated constraints (temp.constr.decl 12.4.2) are the conjunction of the associated constraints of `g` and a constraint that is satisfied if and only if the arguments of `A` are deducible (see below) from the return type.

- If `f` is a copy deduction candidate (over.match.class.deduct 11.3.1.8), then `f'` is considered to be so as well.
- If `f` was generated from a *deduction-guide* (over.match.class.deduct 11.3.1.8), then `f'` is considered to be so as well.
- The *explicit-specifier* of `f'` is the *explicit-specifier* of `g` (if any).

The arguments of a template `A` are said to be deducible from a type `T` if, given a class template

```
template <typename> class AA;
```

with a single partial specialization whose template parameter list is that of `A` and whose template argument list is a specialization of `A` with the template argument list of `A` (temp.dep.type), `AA<T>` matches the partial specialization.

Initialization and overload resolution are performed as described in (dcl.init 9.3) and (over.match.ctor 12.3.1.3), (over.match.copy 12.3.1.4), or (over.match.list 12.3.1.7) (as appropriate for the type of initialization performed) for an object of a hypothetical class type, where the ~~selected functions and function templates~~ *guides of the template named by the placeholder* are considered to be the constructors of that class type for the purpose of forming an overload set, and the initializer is provided by the context in which class template argument deduction was performed. As an exception, the first phase in 12.3.1.7 (considering initializer-list constructors) is omitted if the initializer list consists of a single expression of type *cv* `U`, where `U` is a specialization of `C` the class template for which the placeholder names a specialization or a class derived ~~from a specialization of `C`~~therefrom. If the function or function template was generated from a constructor or *deduction-guide* that had an *explicit-specifier*, each such notional constructor is considered to have that same *explicit-specifier*. All such notional constructors are considered to be public members of the hypothetical class type.

Add the following example to the end of over.match.class.deduct

```
B b{(int*)0, (char*)0}; // OK, deduces B<char*>
```

— *end example*]

[*Example:*

```
template <class T, class U> struct C {
  C(T, U);   // #1
};
template<class T, class U>
C(T, U) -> C<T, std::type_identity_t<U>>; // #2

template<class V>
using A = C<V *, V *>;
template<std::Integral W>
using B = A<W>;

int i{};
double d{};
A a1(&i, &i); // Deduces A<int>
A a2(i, i);   // Ill-formed: cannot deduce V * from i
A a3(&i, &d); // Ill-formed: #1: Cannot deduce (V*, V*) from (int *, double *)
                              // #2: Cannot deduce A<V> from C<int *, double *>
B b1(&i, &i); // Deduces B<int>
B b2(&d, &d); // Ill-formed: cannot deduce B<W> from C<double *, double *>
```

Possible exposition only implementation of the above procedure:

```
//The following concept ensures a specialization of A is deduced
template <class> class AA;
template <class V> class AA<A<V>> {};
template <class T> concept deduces_A = requires { sizeof(AA<T>); };

// f1 is formed from the constructor #1 of C
// generating the following function template
```

```
template<T, U>
auto f1(T, U) -> C<T, U>;

// Deducing arguments for C<T, U> from C<V *, V*> deduces T as V * and U as V *

// f1' is obtained by transforming f1 as described by the above procedure
template<class V> requires deduces A<C<V *, V *>>
auto f1_prime(V *, V*) -> C<V *, V *>;

// f2 is formed the deduction-guide #2 of C
template<class T, class U> auto f2(T, U) -> C<T, std::type_identity_t<U>>;

// Deducing arguments for C<T, std::type_identity_t<U>> from C<V *, V*> deduces T as V *

// f2' is obtained by transforming f2 as described by the above procedure
template<class V, class U>
  requires deduces A<C<V *, std::type_identity_t<U>>>
auto f2_prime(V *, U) -> C<V *, std::type_identity_t<U>>;

// The following concept ensures a specialization of B is deduced
template <class> class BB;
template <class V> class BB<B<V>> {};
template <class T> concept deduces_B = requires { sizeof(BB<T>); };

// The guides for B derived from the above f' and f1' for A are
template<std::Integral W>
  requires deduces_A<C<W *, W *>>
           && deduces_B<C<W *, W *>>
auto f1_prime_for_B(W *, W *) -> C<W *, W *>;

template<std::Integral W, class U>
  requires deduces_A<C<W *, std::type_identity_t<U>>>
           && deduces_B<C<W *, std::type_identity_t<U>>>
auto f2_prime_for_B(W *, U) -> C<W *, std::type_identity_t<U>>;
```

— end example]

Change the following row in Table 17 (tab:cpp.predefined.ft)

`__cpp_deduction_guides`	~~`201703L`~~`????`